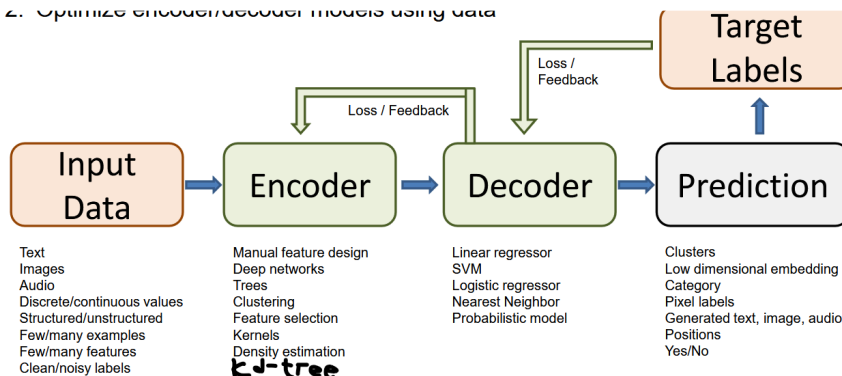


ML

1. Machine learning is all about making decisions.
2. The whole machine learning problem
 1. Data preparation
 1. Collect and curate data
 2. Annotate the data (for supervised problems)
 3. Split your data into train, val, test sets
 2. Algorithm and model development
 1. Design methods to extract features from the data
 2. Design machine learning model and identify key parameters and loss
 3. Train, select params, and evaluate your designs using the val set
 3. Final evaluation using test set
 4. Integrate into your application
3. Optimize encoder/decoder models using data



- 3.
4. Supervised learning, regression
 1. $f(X) = P(Y|X)$
5. dataset X , marginal distribution, $\mu(x) = P(X)$
6. defined so called regression function, $\eta(x) = E[Y|X = x]$, read: expectation of the prediction Y given $X = x$.
7. In the special case of classification, the regression function can be rewritten as $\eta(x) = 0 \cdot P(Y = 0, X = x) + 1 \cdot P(Y = 1|X = x) = P(Y = 1|X = x)$, but this is just regression! or being reduced to a regression problem.
8. If the expectations of the predictions, $\eta(x)$ is close to 0.5, i.e. if η is close to 1, then classifying x is easy if η is close to 0.5, then classifying x is difficult.
9. The probability distribution $P(X, Y)$ is uniquely determined by μ and η .
10. Intuition (discrete case): We can rewrite $P(X = x, Y = 1) = P(Y = 1|X = x)P(X = x) = \eta(x)\mu(x)$ and similarly, $P(X = x, Y = 0) = P(Y = 0|X = x)P(X = x) = (1 - \eta(x))\mu(x)$
11. So we can express the probability of any event (x, y) in terms of η and μ .
12. For formal proof for the general case: see the book of Devorye, Gyorf, Lugosi, first pages.
13. Explicit form of Bayes classifier under 0-1 loss.
 1. Note: now assume the classifier $f(X) = P(Y|X = x)$ gives either 0 or 1 or $\{0, 1\}$.
 2. The risk of a classifier under the 0-1 loss counts how often the classifier fails, that is $R(f) = E(l(X, Y, f(X))) = E(\mathbf{1}_{f(X) \neq Y}) = P(f(X) \neq Y)$.
 3. The bayes classifier f^* was defined as the classifier that minimizes the **true** risk. This is a hypothetical classifier function.
 4.
$$f^0(x) \rightarrow \begin{cases} 1, & \text{if } \eta(x) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$$
 5. basically 0.5 is an optimal threshold, if set higher than 0.5 you get FN if set lower, you get FP.
 6. f^0 is the Bayes classifier. The theorem shows that $f^0 = f^*$ (why?). Consequence: in the particular case of classification with the 0-1 loss, we have an explicit formula for the Bayes Classifier. In practice we do not know f^* why?
 7. Theorem: f^0 is the Bayes classifier. Consider classification with 0-1 loss. let $f : x \rightarrow \{0, 1\}$ be any measurable classifier and f^0 the classifier defined above. Then $R(f) \geq R(f^0)$. Then why don't we just find the Bayes classifier for all problems and go home?

8. Answer: we can only compute the Bayes Classifier if we know the underlying probability distribution, namely we need to evaluate the function η , we can evaluate η only if we know the underlying distribution, but we cannot get the underlying distribution if only based on a limited set of data.

9. Proof:

10. Step 1: consider any fixed classifier $f : x \rightarrow \{0, 1\}$ and compute its error probability at some fixed point x : loss:

$$\begin{aligned} P(f(x) \neq Y|X = x) &= 1 - P(f(x) = Y|X = x) \\ &= 1 - P(f(x) = 1, Y = 1|X = x) - P(f(x) = 0, Y = 0|X = x) \\ &= 1 - 1_{f(x)=1}P(Y = 1|X = x) - 1_{f(x)=0}P(f(x) = 0|X = x) \\ &= 1 - 1_{f(x)=1}\eta(x) - 1_{f(x)=0}(1 - \eta(x)) \end{aligned}$$

11. Step 2: Now compare the pointwise error of any particular classifier f to the one of f^0 ,

$$P(f(X) \neq Y|X = x) - P(f^0(X) \neq Y|X = x).$$

12. Plugging in the equation from step 1, and using the property $1_{x=0} = 1 - 1_{x=1}$ or $1_{x<0} = 1 - 1_{x \geq 1}$,

$$\begin{aligned} P(f(X) \neq Y|X = x) - P(f^0(X) \neq Y|X = x) &= \\ 1 - 1_{f(x)=1}\eta(x) - (1 - 1_{f(x)=1})(1 - \eta(x)) - 1 + 1_{f^0(x)=1}\eta(x) + (1 - 1_{f^0(x)=1})(1 - \eta(x)) &= \\ (1_{f^0(x)=1} - 1_{f(x)=1})(2\eta(x) - 1) \end{aligned}$$

13. if $1_{f^0(x)=1} = 1 \Rightarrow \eta(x) \geq 0.5$ then the first term $(1_{f^0(x)=1} - 1_{f(x)=1})$ is +, the second term $(2\eta(x) - 1)$ is +.

14. if $1_{f^0(x)=1} = 0 \Rightarrow \eta(x) < 0.5$ then the first term $(1_{f^0(x)=1} - 1_{f(x)=1})$ is -, the second term $(2\eta(x) - 1)$ is -.

15. Hence, loss $= (2\eta(x) - 1)(1_{f^0(x)=1} - 1_{f(x)=1}) \geq 0$.

16. Step 3: we have seen that for all fixed values of x , $P(f(x) \neq Y|X = x) \geq P(f^0(x) \neq Y|X = x)$ where f^0 is the Bayes classifier. Because this holds for any individual value of x , it also holds in expectation over all x . This implies $R(f) \geq R(f^0)$.

14. Summary:

1. If we work with 0-1 loss, one don't need to 'learn' the optimal classifier if we know the function $\eta(x)$. However in real life, we don't know the function $\eta(x)$.
2. For many other loss functions one can also explicitly compute the optimal classifier, one such example is regression with squared loss.
3. Problem in practice is we don't know the underlying prior distribution (population data) $\mu(x)$ and the regression function $\eta(x)$.

15. Note: Linear regression

1. Why not just use matrix inversion? because matrix inversion scales badly with number of features d . With n rows and d features compute is $O(d^3 + nd^2)$ and overhead space is $O(d^2)$. Iterative (gradient descent) methods are better for larger datasets. For small d matrix inversion scales just as well as gradient descent.

16. Empirical Risk Minimization (ERM)

1. $R_n(f) = \frac{1}{n} \sum_{i=1}^n l(X_i = x_i, Y_i = y_i, f(X = x_i))$
2. Define a set $\mathcal{F}$ of functions from $X \rightarrow Y$.
3. Within these functions, choose one that has the smallest empirical risk.

$$1. f_n = \arg \min_{f \in \mathcal{F}} R_n(f)$$

2. (if minimum does not exist, only infimum exists, then it will be that converges to a function f_n . The function sequence classifier functions may not even be unique, but in practice it does not pose any problems).

4. Estimation vs approximation (model) error

5. Underfitting means $\mathcal{F}$ is too small:

1. small estimation error.
2. large model error (approximation error).

6. Overfitting means $\mathcal{F}$ is too large:

1. large estimation error.
2. small model error (approximation error).

7. Bias-variance trade-off

1. Bias term: how much difference between f_n and f^* (same flavour as model error).
2. Variance term: the variance of the RV f_n (estimation error).

8. ERM is a straightforward learning principle.

9. The key to success/failure of ERM is to choose a 'good' function class $\mathcal{F}$.

10. Finding the minimizer of the 0-1 loss can be challenging as it is often NP-hard. In practice, we use convex relaxation of the 0-1 loss function, see below.

11. Approach:

1. Define regularized risk: $R_{reg,n}(f) = R_n(f) + \lambda\Omega(f)$. Here $\lambda > 0$ is called a regularizer constant.
2. Then choose $f \in \mathcal{F}$ to minimize the regularized risk.
3. Regularizer, $\Omega : \mathcal{F} \rightarrow \mathbb{R}_{>0}$, that measures how 'complex' a function is,
4. Examples:
 1. $\mathcal{F}$ = polynomials, Ωf = degree of the polynomial f .
 2. $\mathcal{F}$ = differential functions, $\Omega(F)$ = maximal slope.
5. if λ very small: overfitting, high bias error.
6. if λ very large: underfitting, high variance error.

17. when we talk about a distribution

1. model
2. parameter
3. random variable

18. distribution models (based on study order):

1. Bernoulli, RV: $X = \begin{cases} 1, & \text{with probability } p \\ 0, & \text{with probability } q=1-p \end{cases}$, example: coin tossing $\rightarrow$ multinoulli (categorical), RV: 1 to k , example: dice casting, word drawing. Softmax function is often used to predict the probabilities associated with multinoulli distribution.
2. Binomial, RV: the results of n i.i.d. Bernoulli distributions, example: for coin tossing, getting exactly k successes in n independent trials $\rightarrow$ multinomial: the results of n i.i.d. categorical distribution, example: cast a dice n times.
3. Uniform, RV: a **continuous** real number $\rightarrow$ vector RV, like a elevated 2D plane in a 3D plot.
4. Normal/Gaussian, RV: a **continuous** real number, bell shaped, with the central point is the most probable value $\rightarrow$ vector RV, like a sombrero hat in a 3D plot.
5. Poisson, RV: number of events in a fixed interval of time (or space).
6. Geometric, RV: the amt of independent failures k until the first success, each with success probability $p \rightarrow$ Exponentials, RV: continuous analogue of geometric distribution, the amt of time until a next event occurs (success).
7. Gamma, RV: the wait time until the k -th event occurs.
8. Chi-squared, RV: sum of squared of independent standard normal RVs.
9. Beta, RV: continuous value between 0 and 1, i.e. $[0, 1] \in \mathbb{R}$.

19. Bayesian decision rule

1. Given the height of a person we want to predict the gender.
2. RV: Y: male or female, RV: X: height.
3. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sat Apr 11 20:17:07 2026

@author: reina
"""

import numpy as np
import matplotlib.pyplot as plt

# probability density function or likelihood
def gaussian_pdf(x, mu, var):
    a = 1 / np.sqrt(2 * np.pi * var)
    b = - (x - mu)**2 / var
    return a * np.exp(b)

# RV: X: height
# RV: Y: male or female
# want to predict P(Y | X)

# Define likelihood
x = np.linspace(0, 220, num=100)
mu1 = 175; var1 = 50
mu2 = 162; var2 = 50
likelihood1 = gaussian_pdf(x, mu1, var1)
likelihood2 = gaussian_pdf(x, mu2, var2)

# Define priors
prior1 = 0.3
prior2 = 1 - prior1

# Compute marginal
marginal = likelihood1 * prior1 + likelihood2 * prior2
```

```

# Compute posteriors
posterior1 = likelihood1 * prior1 / marginal
posterior2 = likelihood2 * prior2 / marginal

fig = plt.figure(figsize=(16, 12))

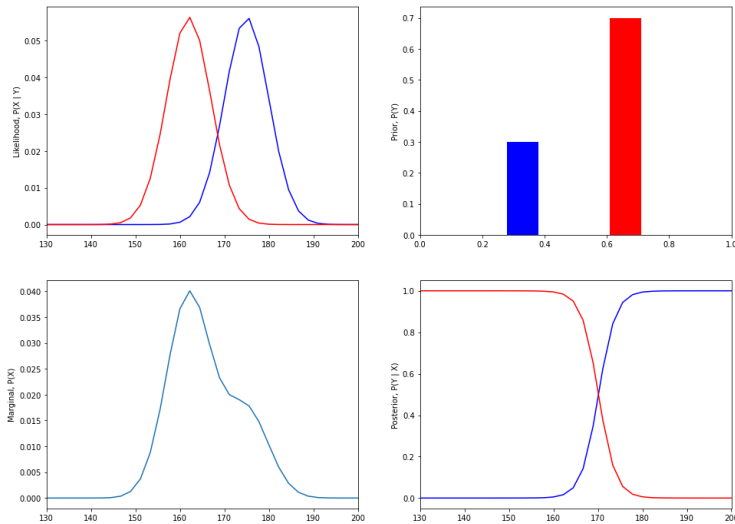
# Plot likelihood
ax1 = fig.add_subplot(221)
ax1.plot(x, likelihood1, color='b')
ax1.plot(x, likelihood2, color='r')
ax1.set_xlim(130, 200)
ax1.set_ylabel('Likelihood, P(X | Y)')

# Plot priors
ax2 = fig.add_subplot(222)
ax2.bar(0.33, prior1, width=0.1, color='b')
ax2.bar(0.66, prior2, width=0.1, color='r')
ax2.set_xlim(0, 1)
ax2.set_ylabel('Prior, P(Y)')

# Plot marginal
ax3 = fig.add_subplot(223)
ax3.plot(x, marginal)
ax3.set_xlim(130, 200)
ax3.set_ylabel('Marginal, P(X)')

# Plot posterior
ax4 = fig.add_subplot(224)
ax4.plot(x, posterior1, color='b')
ax4.plot(x, posterior2, color='r')
ax4.set_xlim(130, 200)
ax4.set_ylabel('Posterior, P(Y | X)')

```



Linear regression

1. Linear regression is a supervised parametric model. It assumes the data to be a Gaussian distribution.
2. Matrix form:
 1. X: shape: (N, d+1)
 2. w: shape: (d+1, 1)
 3. y: shape: (N,)
3. prediction $\hat{y} = Xw$
4. $L = \sum_{i=1}^N (y - \hat{y})^2 = \sum_{i=1}^N (y - Xw)^2$
5. Direct solution: $\frac{\partial L}{\partial w} = 0 \Rightarrow w = (X^T X)^{-1} X^T y$

Perceptron

Logistic regression

1. Logistic regression is a supervised parametric model for classification. It assumes the data to be Bernoulli distributed.
2. Calculate score: $\vec{w}^T \vec{x}$, if score > 0 , the sigmoid function will return a value ≥ 0.5 else a value < 0.5 .
3. Steps:
 1. compute distance to the hyperplane via dot product, $d = w^T x$, since w is the vector normal to the hyperplane.
 2. pass the distance through a sigmoid function that outputs 0 or 1.

3. compute loss using cross-entropy loss or binary cross entropy loss.
4. Although the sigmoid function is nonlinear, the classification itself is a linear operation since it has a linear hyperplane.
5. For positive class + as $y = 1$ and negative class - as $y = -1$:
 1. Vector form:
 1. $\vec{x}$: shape: (d+1, 1), +1 for bias,
 2. $\vec{w}$: shape: (d+1, 1),
 3. label $\vec{y}$ is a scalar 1 or -1.
 4. $score = \vec{w}^T \vec{x}$.
 2. Matrix form:
 1. X: shape: (N, d+1), where N: batch_size, d=embed_dim
 2. $\vec{w}$: shape: (d+1, 1)
 3. y: shape: (N,)
 3. Sigmoid function: $\sigma(\vec{w}^T \vec{x}) = \frac{e^{\vec{w}^T \vec{x}}}{1 + e^{\vec{w}^T \vec{x}}} = \frac{1}{1 + e^{-\vec{w}^T \vec{x}}}$.
 4.

$$y_i = \begin{cases} 1, & \text{if } \sigma(\vec{w}^T \vec{x}_i) \geq 0.5 \\ -1, & \text{if } \sigma(\vec{w}^T \vec{x}_i) < 0.5 \end{cases}$$
 5. Using only one weight vector $\vec{w}$ for prediction. Note: In the general case e.g. multi-class classification using Softmax, we usually have weights for each class i.e. $w_1, w_2, \dots, w_n$. Also the Sigmoid function is a special case of Softmax:
 Given a binary classification case, $f_1 = \frac{e^{w_1^T x}}{e^{w_1^T x} + e^{w_2^T x}}$, and $f_2 = \frac{e^{w_2^T x}}{e^{w_1^T x} + e^{w_2^T x}}$. Then Multiply the numerator and denominator of f_1 by $e^{-w_1^T x}$, and set $w = w_1 - w_2$, we'll then arrive at the Sigmoid function.
 6. Next we'll derive the gradient update rule for SGD using the vector form.
 7. Properties of Sigmoid function: symmetric: $1 - \sigma(x) = \sigma(-x)$ and $\sigma'(x) = \sigma(x)\sigma(-x)$.
 8. If $y_i = 1 \Rightarrow P_{\vec{w}}(y_i | \vec{x}_i) = \sigma(\vec{w}^T \vec{x}_i)$.
 9. If $y_i = -1 \Rightarrow P_{\vec{w}}(y_i | \vec{x}_i) = 1 - \sigma(\vec{w}^T \vec{x}_i) = \sigma(-\vec{w}^T \vec{x}_i)$.
 10. Thus $l(\vec{w}, \vec{x}_i, y_i) = -\log(\sigma(y_i \vec{w}^T \vec{x}_i))$
 11. Let $u = y_i \vec{w}^T \vec{x}_i$, $\frac{\partial l}{\partial u} = -\frac{1}{\sigma(u)} \sigma(u) \sigma(-u) = \sigma(-u)$, $\frac{\partial u}{\partial \vec{w}} = y_i \vec{x}_i$. Hence, $\frac{\partial l}{\partial \vec{w}} = -\sigma(-u) y_i \vec{x}_i$.
 12. SGD update: $\vec{w} = \vec{w} + \eta y_i \sigma(-y_i \vec{w}^T \vec{x}_i) \vec{x}_i$.
6. For positive class + as $y = 1$ and negative class - as $y = 0$:
 1. Using a simplified example: $\vec{x} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \end{pmatrix}$, $\vec{w} = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \end{pmatrix}$. Then $\hat{y} = \frac{e^{w_0 + w_1 x_1 + w_2 x_2}}{1 + e^{w_0 + w_1 x_1 + w_2 x_2}}$
 2. Note: for general case, we have $w_1 = \begin{pmatrix} w_0^{(1)} \\ w_1^{(1)} \\ w_2^{(1)} \end{pmatrix}$ and $w_2 = \begin{pmatrix} w_0^{(2)} \\ w_1^{(2)} \\ w_2^{(2)} \end{pmatrix}$. But for the binary case we suffice to use just 1 weight vector.
 3. It can be shown that, treating derivatives in scalar form: $\frac{\partial \hat{y}}{\partial w_0} = \hat{y}(1 - \hat{y})$, $\frac{\partial \hat{y}}{\partial w_1} = x_1 \hat{y}(1 - \hat{y})$, $\frac{\partial \hat{y}}{\partial w_2} = x_2 \hat{y}(1 - \hat{y})$.
 4. $L = -(y \log \hat{y} + (1 - y) \log (1 - \hat{y}))$
 5. $\frac{\partial L}{\partial \hat{y}} = -(\frac{y}{\hat{y}} - (1 - y) \frac{1}{1 - \hat{y}}) = -\frac{y - \hat{y}}{\hat{y}(1 - \hat{y})}$
 6. $\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_0} = -\frac{y - \hat{y}}{\hat{y}(1 - \hat{y})} \hat{y}(1 - \hat{y}) = \hat{y} - y$
 7. $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_1} = -\frac{y - \hat{y}}{\hat{y}(1 - \hat{y})} x_1 \hat{y}(1 - \hat{y}) = (\hat{y} - y) x_1$
 8. $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2} = (\hat{y} - y) x_2$
 9. SGD gradient update: In vector form: $\vec{w} = \vec{w} - \frac{\eta}{m} \vec{x}^T (\hat{\mathbf{y}} - \mathbf{y})$. Note: the m is optional, it arises from taking the average from the loss function given above.

Multi-class logistic regression

SVM

1. The distance of a training point $\vec{v}(x_i)$ to the hyperplane is: $r = \frac{|\vec{w}^T \vec{v}(x_i) + b|}{\|\vec{w}\|}$. Recall the distance is the projection of the vector $\vec{v}(x_i)$ to the vector perpendicular to the hyperplane, $\vec{w}$. Hint: use dot product formula of two vectors.
2. Now, in SVM we impose the constraint $|\vec{w}^T \vec{v}(x_i) + b| = 1$. So now we have $r = \frac{1}{\|\vec{w}\|}$.
3. As the hyperplane changes, the distance r will change accordingly. The goal of SVM is to find of all the possible hyperplanes, the one that has the maximum margin $2r$. This equates to maximizing $2r = \frac{2}{\|\vec{w}\|}$ or minimizing $\frac{1}{2} \|\vec{w}\|$.
4. In practice we only use soft margin SVM.

- Slack variable defines the violation, $\zeta_i = \max(0, 1 - y_i(\vec{w}^T \vec{v}(x_i) + b))$. For all training instances, each training instance has a slack variable, has a violation.
- So we want to maximize $\frac{1}{\|\vec{w}\|}$ or minimize $\|\vec{w}\|$ but also minimize the slack variable as well ζ_i . If I want to minimize the two things, we no longer have constraints. Because the constraints is already expressed in the slack variables. So we want to minimize the violation to the score plus our original score margin:
 $(\vec{w}, b) = \arg \min_{(\vec{w}, b)} C \sum_i \max(0, 1 - y_i(\vec{w}^T \vec{v}(x_i) + b)) + \frac{1}{2} \|\vec{w}\|^2$. C times the total of the slack variable. C is a constant to weight between the importance between the violation and the original margin. So this is the new training objective, it becomes an unconstrained minimization problem.

Naive Bayes

- The general term Naive Bayes refers to the strong independence assumptions in the model, rather than the particular distribution of each feature.
- A Naive Bayes model assumes that each of the features it uses are conditionally independent of one another given some class.
- More formally, to calculate the probability of observing features f_1 through f_n , given some class c , under the NB assumption the following holds: $p(f_1, \dots, f_n | c) = \prod_{i=1}^n p(f_i | c)$. This means that when I want to use a NB model to classify a new example, the posterior probability is much simpler to work with: $p(c | f_1, \dots, f_n) \propto p(c) p(f_1 | c) \dots p(f_n | c)$. Of course these assumptions of independence are rarely true, which may explain why some have referred to the model as the "Idiot Bayes" model, but in practice NB have performed surprisingly well, even on complex tasks where it is clear that the strong independence assumptions are false.
- BernoulliNB, CategoricalNB, MultinomialNB in sklearn.
- Parameters are key to control the separator
- Text classification Bernoulli NB example:

$$1. \phi = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{|V|} \end{bmatrix}, \vec{\theta}_{sports} = \begin{bmatrix} \log(P(goal|sports)) \\ \log(P(fans|sports)) \\ \vdots \\ \log(P(stock|sports)) \\ \log(P(loan|sports)) \\ \log(P(CEO|sports)) \end{bmatrix}$$

- $f_1 = \#goal \in d$
- $\log P(c = sports | d) = \vec{\theta}_{sports} \cdot \vec{\phi} + \log P(c = sports)$.
- its a linear model, very similar to perceptron.
- Parameter for NB is $\log P(w | c)$.
- Both SVM and perceptron are discriminative models.
- NB is a generative model, calculating joint probabilities of inputs and outputs.
- All three models are linear models.
- $\log P(c)$ says, if a sports document is much more than finance document, then you will have a higher probability to see sports document (this is a useful pattern, how can we add this to the perceptron? by adding a bias term).

$$10. \phi = \begin{bmatrix} 1 \\ f_1 \\ f_2 \\ \vdots \\ f_{|V|} \end{bmatrix}, \vec{\theta}_{sports} = \begin{bmatrix} \log P(sports) \\ \log(P(goal|sports)) \\ \log(P(fans|sports)) \\ \vdots \\ \log(P(stock|sports)) \\ \log(P(loan|sports)) \\ \log(P(CEO|sports)) \end{bmatrix}$$

- $f_1 = \#goal \in d$
- $\log P(c = sports | d) = \vec{\theta}_{sports} \cdot \vec{\phi}$.
- NB cannot integrate bigram features, bigram patterns.

Decision Trees

- Decision trees are non-parametric supervised learning method for classification and regression.
- CART vs ID3 vs C4.5.

K-means:

- K-means is a non-parametric clustering method.

2. Special case of EM, special case of Autoencoder

3. discrete distribution

4. K-means:

1. randomly place K centroids, where K is the number of clusters we want.
2. these centroids act like gravity pulling closer points into their orbit.
3. the algorithm calculates the distance of each data point to the centroids assigning data points to the closest one forming clusters.
4. now the centroid shift to the average position of all the points in their cluster.
5. the process repeat until the centroids stop moving.

5. Simple and efficient, but need to specify K.

6. Code: k-means.py

```
# -*- coding: utf-8 -*-
"""
Created on Mon Apr 13 21:27:53 2026

@author: reina
"""

import numpy as np

def k_means_clustering(X, k, max_iter=100):
    # Step 1: Initialize cluster centroids randomly
    centroids = X[np.random.choice(range(len(X)), k, replace=False)]

    # Step 2: Repeat until convergence or max_iter reached
    for _ in range(max_iter):
        # Step 2a: Assignment - assign each data point to the nearest cluster centroid
        # (N, D) - (k, 1, D) = (k, N, D) -sum(axis=2)-> (k, N)
        distances = np.sqrt(((X - centroids[:, np.newaxis])**2).sum(axis=2))
        # (N,)
        cluster_assignments = np.argmin(distances, axis=0)
        # Step 2b: Update - compute new centroids basde on the mean of the data points in each cluster
        new_centroids = np.array([X[cluster_assignments == i].mean(axis=0) for i in range(k)])

        # Check for convergence
        if np.all(centroids == new_centroids):
            break

        centroids = new_centroids

    return centroids, cluster_assignments

# def k_means_clustering_naive(X, k, max_iter=100):
#     centroids = X[np.random.choice(range(len(X)), k, replace=False), :]
#     N, D = X.shape
#     for _ in range(max_iter):
#
#         cluster_assignments = []
#         for i in range(N):
#             distances = []
#             for j in range(k):
#                 dist = np.sqrt(np.sum((X[i, :] - centroids[j])**2))
#                 distances.append(dist)
#             distances = np.array(distances)
#
#             cluster = np.argmin(distances, axis=0)
#             cluster_assignments.append(cluster)
#
#         cluster_assignments = np.array(cluster_assignments)
#
#         new_centroids = np.array([X[cluster_assignments == m].mean(axis=0) for m in range(k)])
#
#         if np.all(new_centroids == centroids):
#             break
#
#         centroids = new_centroids
#
#     return centroids, cluster_assignments

if __name__ == '__main__':
    # Example usage
    X = np.array([[2, 3], [3, 4], [5, 4], [7, 2], [8, 1], [9, 0]])
    k = 2

    centroids, cluster_assignments = k_means_clustering(X, k)
    # centroids, cluster_assignments = k_means_clustering_naive(X, k)
    print("Final centroids:")
    print(centroids)
```

```
print("Cluster assignments:")
print(cluster_assignments)
```

Precision, Recall and F1 score

1. How to picture Precision: Given pictures of cats, the model predicts some images as images of dogs. Then the model's precision for dogs will decrease as it is given by: $TP / (TP + FP)$.
2. How to picture Recall: Given pictures of cats, say the number of instances the model predicted a picture as a cat as True Positives (TP), denote the number of instances the model incorrectly predicts an image of a cat as a non-cat as False Negative (FN). Then, the ratio $TP / (TP + FN)$ is recall.
3. F1 score is the harmonic mean of precision and recall.

References:

1. CS441 UIUC, Derek Hoes, 2025.
2. Statistical Machine Learning, Ulrike Von Luxburg, Tübingen, 2024.
3. CS444: Deep learning for Computer Vision, Linear Classifiers, Saurabh Gupta.
4. [Difference between naive Bayes and multinomial naive Bayes](#)